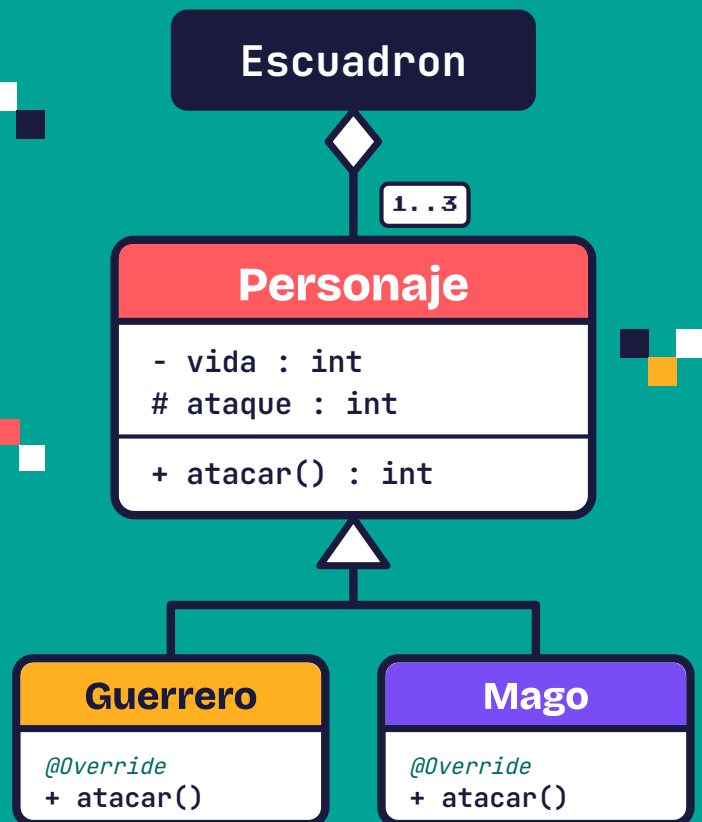


Clase maestra

Taller gamificado y 100 % guiado de programación orientada a objetos en Java

0	Arranque
1	La forja de héroes
2	La bóveda del gremio
3	El linaje
4	La arena de los mil rostros
5	El escuadrón sellado
J	Batalla contra el Dragón Null

**3 h**

de juego en 9 sesiones cronometradas

5+1

niveles y una batalla contra el jefe final

1180

XP en disputa entre retos, bonus y modo difícil

3-4

jugadores por equipo con roles rotativos

Este cuadernillo es tu mapa: cada página dice qué escribir, qué debería aparecer en pantalla y qué hacer si aparece otra cosa. Vas a construir un juego de rol en Java, clase por clase, en equipo y ganando puntos.

Lo que vas a lograr

#	Al terminar el taller podrás...	Nivel cognitivo	Dónde
1	Distinguir clase y objeto , y darles estado y comportamiento con atributos y métodos.	Comprender	Nivel 1
2	Construir objetos válidos con constructores y proteger su estado con <code>private</code> , getters y setters.	Aplicar	Nivel 2
3	Reutilizar código con herencia usando <code>extends</code> , <code>super</code> , <code>protected</code> y <code>@Override</code> .	Aplicar	Nivel 3
4	Predecir qué versión de un método se ejecuta con polimorfismo y distinguir sobrecarga de sobrescritura.	Analizar	Nivel 4
5	Modelar relaciones “tiene un” con agregación y sellar reglas con <code>final</code> .	Analizar	Nivel 5
6	Integrar todo en una batalla por turnos y demostrar con una tabla de rondas que tu estrategia gana.	Crear	Jefe final

Tu equipo de juego

- ✓ Un computador por equipo con un compilador de Java en línea (OnlineGDB, Programiz o JDoodle, eligiendo el lenguaje Java). No hay que instalar nada.
- ✓ Alternativa: IntelliJ IDEA, VS Code con Extension Pack for Java o NetBeans, con JDK 17 o superior.
- ✓ Este cuadernillo impreso o en pantalla, lápiz, una regla y un cronómetro visible para todo el salón.

La regla del archivo único

1. Todo el programa vive en un solo archivo llamado `Main.java`.
2. `public class Main` va **primero**, con el método `main` adentro.
3. Las demás clases van debajo, fuera de las llaves de `Main` y **sin** la palabra `public` delante de `class`.
4. Si el compilador marca un error, lee el número de línea: `Main.java:12` significa “mira la línea 12”.

Cómo está construido y cómo leerlo

Cada nivel sigue la misma ruta: el docente **modela** un ejemplo base, lo recorren con **PRIMM** (predecir, ejecutar, investigar, modificar y crear) y luego vienen predicciones, retos y cacería de bugs. Todo el código fue compilado y ejecutado antes de imprimirse. Estas señales se repiten en todo el cuadernillo:



Ejemplo base

Código resuelto y explicado línea por línea.



Salida y errores

Lo que debe aparecer. En rojo, lo que dice el compilador.



Reto

XP a la izquierda; marquen casillas solo si funciona.



Cazabugs

Código roto a propósito: lean antes de ejecutar.



Checkpoint

Autoevaluación honesta. “Ayuda” no resta XP.



Oráculo

Consulta a la IA, anotada en el Anexo B.

Reglas del juego

Manual del jugador

Juegan equipos de 3 o 4 personas que forman un gremio de forjadores de objetos. Ganan XP por pensar, no solo por teclear. Al final, el equipo con más XP se lleva el título de Gran maestro POO.

Roles del equipo

Los roles rotan al comenzar cada nivel: Piloto pasa a Navegante, Navegante a Cronista, Cronista a Auditor y Auditor a Piloto. En equipos de 3, el Cronista también es Auditor de IA.



Piloto

Tiene el teclado y escribe el código. No decide solo: escucha al Navegante.

"Yo tecleo, tú me guías."



Navegante

Lee las instrucciones en voz alta, dicta y revisa llaves, puntos y comas.

"Ojo, ese constructor no lleva void."



Cronista

Escribe predicciones, diagramas y tablas en el cuadernillo antes de ejecutar.

"Primero lo anoto, luego lo corremos."



Auditor de IA

El único que consulta la IA cuando el equipo activa el comodín. Verifica y explica.

"La IA dijo esto, y significa..."

Cómo se gana XP

Acción	XP
Predicción correcta, escrita antes de ejecutar	+10
Reto completado que compila y funciona	+20 a 40
Bug encontrado y corregido en Cazabugs	+10 c/u
Explicar un reto en voz alta al docente (1 por nivel)	+15
Bonus "sin Oráculo": terminar el nivel sin usar la IA	+25
Fases del jefe final	hasta 200



La regla de oro: predice antes de ejecutar

Una predicción solo cuenta si está escrita en el cuadernillo **antes** de ejecutar. Equivocarse no resta puntos: es la mejor forma de descubrir qué no entendías.



Código que nadie explica, no suma

El docente puede señalar cualquier línea y preguntar qué hace. Si nadie del equipo puede explicarla, ese reto queda en 0 XP hasta que alguien pueda.

Rangos

0-299 Aprendiz del new 	300-599 Forjador de objetos 	600-899 Arquitecto de clases 	900+ Gran maestro POO
--	---	--	---

Marcador del equipo

El Cronista suma el XP al cerrar cada etapa. El docente lo pasa a la tabla de clasificación del Anexo A.

Equipo	S0	N1	N2	N3	N4	N5	Jefe	Cierre	Total

Cada equipo recibe tres comodines para todo el taller. Úsenlos con estrategia: el Oráculo gastado en un nivel les quita el bonus “sin Oráculo” de ese nivel.



Oráculo (IA)

Tres consultas a una IA en todo el taller. Solo las hace el Auditor.

1. Pregunta por conceptos o pistas, no por la solución completa.
2. Anota la pregunta en la bitácora del Oráculo (Anexo B).
3. Si la IA entrega código, el equipo predice su salida antes de ejecutarlo.
4. Explica la respuesta con tus palabras al resto del equipo.



Llamada al docente

Una pista directa del docente, sin perder XP. Úsenla cuando lleven más de 3 minutos atascados con el mismo error.



Espía

Un minuto para mirar la pantalla de otro equipo. Solo mirar: nada de fotos ni copiar.

La IA es un power-up, no el piloto automático

Pueden usar inteligencia artificial y aun así disfrutar el taller. El truco está en el diseño: la mayoría de los puntos se ganan con acciones que ninguna IA puede hacer por ustedes en tiempo real y dentro del salón.

Lo que la IA puede hacer	Lo que solo tu equipo puede hacer	XP en juego
Escribir una clase Java que compile	Predecir en papel qué imprime antes de ejecutarla	+10 por predicción
Explicar qué es el polimorfismo	Explicar su propio código cuando el docente señala una línea	+15 por nivel
Resolver un ejercicio genérico	Resolver retos con datos reales: sus cumpleaños, los objetos de su mesa, las medidas del salón	retos personalizados
Dibujar un diagrama UML cualquiera	Dibujar a mano el UML de su código y defenderlo frente al docente	+30 en el reto 3.2
Dar la respuesta a un bug	Encontrarlo antes que los otros equipos, con el cronómetro corriendo	+10 por bug
Proponer una estrategia	Demostrar con una tabla de rondas que su escuadrón vence al dragón	+80 en la fase 5



Preguntas buenas para el Oráculo

“¿Qué diferencia hay entre sobrecarga y sobrescritura? Explícalo con un ejemplo que no sea de personajes.”

“El compilador dice *has private access*. ¿Qué significa, sin darme el código corregido?”



Preguntas que gastan el comodín sin ayudar

“Resuélveme el reto 4.2.”

“Dame el código del jefe final.” Se llevan un código que no pueden explicar, y eso vale 0 XP.

Mapa de la partida

180 minutos

Nueve sesiones, un solo cronómetro. El docente proyecta el tiempo restante y actualiza la tabla de clasificación al cerrar cada nivel.

00:00 00:15	Sesión 0. Arranque Equipos, roles, entorno Java y calentamiento	¿Clase u objeto? con los pulgares	30 XP
00:15 00:40	Nivel 1. La forja de héroes <code>class</code> , atributos, <code>new</code> y métodos	Modelado, PRIMM y 3 retos	120 XP
00:40 01:05	Nivel 2. La bóveda del gremio Constructores, <code>this</code> , <code>private</code> , getters y setters	La vida que nadie puede hackear	150 XP
01:05 01:15	Pausa activa. Guardar partida Polimorfismo humano y ranking provisional	Ejecutar un for con el cuerpo	0 XP
01:15 01:40	Nivel 3. El linaje <code>extends</code> , <code>super</code> , <code>protected</code> y <code>@Override</code>	UML a mano y rompecabezas de código	150 XP
01:40 02:05	Nivel 4. La arena de los mil rostros Polimorfismo, sobrecarga e <code>instanceof</code>	Una orden, muchas respuestas	150 XP
02:05 02:25	Nivel 5. El escuadrón sellado Agregación y <code>final</code>	Mochila real y reglas selladas	120 XP
02:25 02:50	Jefe final. Batalla contra el Dragón Null Todo combinado en un sistema de clases	5 fases y una estrategia por demostrar	200 XP
02:50 03:00	Cierre. Créditos finales Ticket de salida, insignias y premiación	3-2-1 y conteo final	30 XP



Retos marcados como "Extra"

Son para equipos veloces. Si el tiempo del nivel se acaba, pasan al siguiente sin penalización. Siempre pueden volver a ellos al final o en casa.



Guarden la partida al cerrar cada nivel

Cada nivel empieza en un `Main.java` nuevo. Antes de borrar, copien su código en un documento del equipo: en el jefe final lo van a necesitar, y si algo se pierde, el Anexo C trae un punto de guardado.

Sesión 0. Arranque

15 min

4 min Equipos y nombre	2 min Roles	5 min Primer programa	4 min Calentamiento
----------------------------------	-----------------------	---------------------------------	-------------------------------

Paso a paso

1
Forma tu equipo

Reúnanse en grupos de 3 o 4. Inventen un nombre de gremio estilo gamertag, sin espacios ni tildes, porque lo van a usar como texto en el código.
Nombre del gremio: _____

2
Reparte roles

Escriban quién empieza en cada rol. Recuerden que rotan en cada nivel.
Piloto _____ Navegante _____
Cronista _____ Auditor de IA _____

3
Escribe tu primer programa

El Piloto abre el compilador, borra lo que haya y escribe esto. Cambien el texto por el nombre de su gremio y ejecuten.

```
Main.java
1 public class Main {
2     public static void main(String[] args) {
3         String equipo = "LosNullPointers";
4         System.out.println("Hola, somos " + equipo + " y venimos a forjar objetos");
5     }
6 }
7
```

Salida en consola

```
Hola, somos LosNullPointers y venimos a forjar objetos
```

Kit de supervivencia

Pieza	Para qué sirve
<code>public class Main</code>	Todo programa Java vive dentro de al menos una clase.
<code>main(String[] args)</code>	La puerta de entrada: la ejecución empieza aquí.
<code>System.out.println()</code>	Muestra algo en pantalla. Con <code>+</code> unes textos y valores.
<code>int</code> <code>double</code> <code>boolean</code> <code>String</code>	Entero, decimal, verdadero/falso y texto entre comillas.
<code>;</code> y <code>{ }</code>	El punto y coma cierra instrucciones; las llaves abren y cierran bloques.

Calentamiento: ¿clase u objeto?

Una **clase** es un molde general; un **objeto** es una cosa concreta hecha con ese molde. Pulgar arriba si es clase, abajo si es objeto: 10 segundos por palabra y **+6 XP** por acierto.

#	Palabra	Respuesta
1	Estudiante	
2	El computador que usa tu equipo ahora	
3	Canción	
4	La última canción que escuchó el Navegante	
5	Celular	

La forja de héroes

```
class new atributos métodos return
```

Misión

El gremio quiere un juego de rol lleno de héroes, pero nadie quiere escribir diez variables sueltas por cada personaje. La solución de los forjadores es un molde: se escribe una sola vez cómo es un héroe y se fabrican cuantos se quiera con `new`. Tu misión es forjar ese molde y darles vida a sus objetos.

Al terminar este nivel podrás

- ✓ Explicar la diferencia entre una clase y un objeto con un ejemplo propio.
- ✓ Declarar atributos, crear objetos con `new` y usarlos con el operador punto.
- ✓ Escribir métodos `void` y métodos que devuelven un valor con `return`.
- ✓ Predecir qué pasa cuando dos variables apuntan al mismo objeto.

Botín de XP

3 predicciones relámpago	30
Retos 1.1, 1.2 y 1.3	70
Cazabugs (2 bugs)	20
Total del nivel	120
Bonus sin Oráculo y explicación oral	+40

Reloj del nivel

6' Modelado	5' Guiada	3' Predice	9' Retos	2' Bugs
----------------	--------------	---------------	-------------	------------

🔄 **Rota los roles ahora.** Quien era Piloto pasa a Navegante.

Ejemplo base 1: el molde y sus objetos

Nivel 1

Una **clase** es un plano: dice qué datos tiene un héroe y qué sabe hacer. Un **objeto** es un héroe concreto fabricado con ese plano. Una clase, muchos objetos, cada uno con sus propios datos.

► Ejemplo base 1. Clase Heroe

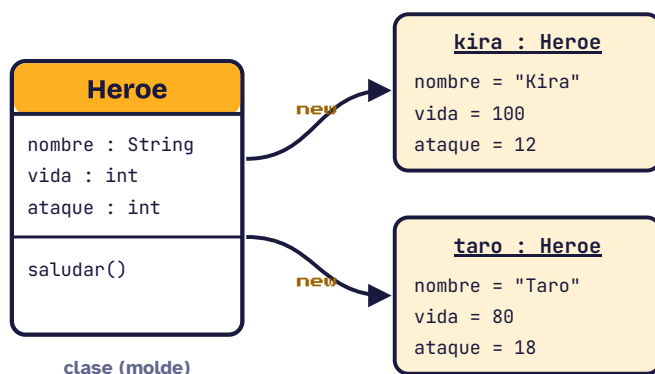
```
Main.java
1 public class Main {
2     public static void main(String[] args) {
3         Heroe kira = new Heroe();
4         kira.nombre = "Kira";
5         kira.vida = 100;
6         kira.ataque = 12;
7
8         Heroe taro = new Heroe();
9         taro.nombre = "Taro";
10        taro.vida = 80;
11        taro.ataque = 18;
12
13        kira.saludar();
14        taro.saludar();
15    }
16 }
17
18 class Heroe {
19     String nombre;
20     int vida;
21     int ataque;
22
23     void saludar() {
24         System.out.println("Soy " + nombre + ", vida " + vida);
25     }
26 }
```

Salida en consola

```
Soy Kira, vida 100
Soy Taro, vida 80
```

Cómo funciona, línea por línea

- L18 `class Heroe` define el molde. Los nombres de clase empiezan con mayúscula y van en singular.
- L19-21 Son los **atributos**: los datos que guarda cada héroe. Cada objeto tiene su propia copia.
- L23 Es un **método**: algo que el héroe sabe hacer con sus propios atributos.
- L3 `new Heroe()` fabrica un objeto en memoria. `kira` guarda una **referencia**: una flecha hacia ese objeto.
- L4 El punto se lee “de”: `kira.nombre` es “el nombre de kira”.
- L13-14 Cada objeto ejecuta el mismo método, pero responde con sus propios datos.



Un molde, dos objetos. En UML, el nombre de un objeto va subrayado y seguido de su clase: `kira : Heroe`.



Anatomía que no se negocia: la palabra `class`, el nombre con mayúscula inicial y las llaves `{ }` que encierran atributos y métodos. Los objetos se fabrican siempre con `new`.

Ejemplo base 2: métodos que hacen y responden

Nivel 1

Unos métodos **hacen** algo y no devuelven nada (`void`). Otros **responden** y entregan un valor con `return`. Los datos que reciben entre paréntesis son **parámetros**.

► Ejemplo base 2. Clase con métodos

>_ Dentro de main

class Heroe (debajo de Main)

```
1 class Heroe {
2     String nombre;
3     int vida;
4     int ataque;
5
6     void recibirDanio(int danio) {
7         vida = vida - danio;
8     }
9
10    boolean estaVivo() {
11        return vida > 0;
12    }
13
14    int golpeCritico() {
15        return ataque * 2;
16    }
17 }
```

main

```
1 Heroe kira = new Heroe();
2 kira.nombre = "Kira";
3 kira.vida = 100;
4 kira.ataque = 12;
5
6 kira.recibirDanio(30);
7 System.out.println(kira.vida);
8 System.out.println(kira.golpeCritico());
9 System.out.println(kira.estaVivo());
```

Salida en consola

```
70
24
true
```

Cómo funciona

L6 `void`: no devuelve nada. `int danio` es un parámetro: se llena con el valor que se envía al llamar.

L7 Cambia el atributo del objeto. Con `kira.recibirDanio(30)`, `danio` vale 30.

L10 Responde una pregunta: `return` entrega `true` o `false` a quien llamó.

L14 Devuelve un número. La llamada `kira.golpeCritico()` se reemplaza por su resultado: 24.

main Un método `void` se usa como instrucción suelta; uno con `return` se usa donde cabe un valor: dentro de `println`, un `if` o una suma.

Diccionario del forjador

Término	Qué es	En nuestro juego
Clase	Plano o molde que describe datos y comportamientos.	Heroe
Objeto	Un ejemplar concreto creado con <code>new</code> ; también se llama <i>instancia</i> .	El héroe al que apunta <code>kira</code>
Atributo	Dato que guarda cada objeto: su estado.	<code>vida</code> , <code>ataque</code>
Método	Acción o pregunta que el objeto sabe responder: su comportamiento.	<code>recibirDanio()</code> , <code>estaVivo()</code>
Referencia	Lo que guarda una variable de objeto: dónde está el objeto.	<code>kira</code> apunta al héroe

Usen el ejemplo base 2. El Cronista escribe cada respuesta antes de que el Piloto ejecute.

- 1**
Predice
 Cambien `kira.recibirDanio(30);` por `kira.recibirDanio(120);`. ¿Qué imprimirán la primera y la tercera línea?
 Nuestra predicción: _____
- 2**
Ejecuta
 El Piloto hace el cambio y ejecuta. ¿Acertaron? _____ ¿Tiene sentido una vida negativa en un juego? _____
- 3**
Investiga
 Copien el método `saludar()` del ejemplo base 1 dentro de la clase. Al final de `main` agreguen `Heroe fantasma = new Heroe();` y `fantasma.saludar();`
 ¿Qué imprime? _____ ¿Por qué aparece `null`? _____
- 4**
Modifica
 Agreguen este método a la clase. Al final de `main` llamen `kira.curar(50);` e impriman `kira.vida`. Con el daño de 120, ¿qué vida muestra? _____


```

1 void curar(int puntos) {
2     vida = vida + puntos;
3 }
      
```
- 5**
Crea
 Pasen a los retos. A partir de aquí escriben su propio código. Guarden la pregunta del fantasma: en el Nivel 2 la resolvemos.



El secreto del fantasma: valores por defecto

Si nadie les da valor, los atributos arrancan con uno por defecto: `int` y `double` en `0`, `boolean` en `false`, y `String` o cualquier objeto en `null`, que significa “no apunta a nada”.

Predicciones relámpago

Sin ejecutar nada. Tres minutos para las tres. Usan la clase `Heroe` del ejemplo base 2 con `saludar()` incluido.

Predicción A	Predicción B	Predicción C
+10 <pre> 1 Heroe a = new Heroe(); 2 a.vida = 50; 3 Heroe b = a; 4 b.vida = 10; 5 System.out.println(a.vida); </pre>	+10 <pre> 1 Heroe x = new Heroe(); 2 x.ataque = 7; 3 x.ataque = x.golpeCritico(); 4 int r = x.golpeCritico(); 5 System.out.println(r); </pre>	+10 <pre> 1 Heroe z = new Heroe(); 2 System.out.println(z.nombre); 3 System.out.println(z.vida); 4 System.out.println(z.estaVivo()); </pre>
Imprime: _____	Imprime: _____	Imprime (3 líneas): _____ _____ _____



Mapa de memoria para la predicción A

Dibujen una cajita por cada variable y un rectángulo por cada objeto creado con `new`. Unan con flechas. ¿Cuántos rectángulos hay en la predicción A? Contar los `new` nunca falla.

30 XP

Reto 1.2

Tu héroe, tus datos

Forjen un `Heroe` con datos reales: `nombre` con las iniciales de todos, `vida` igual a la suma de los días de cumpleaños de los integrantes (si nacieron el 12, el 3 y el 28, la vida es 43) y `ataque` igual al número de letras del nombre del Piloto. Agreguen el método `int golpesParaVencer(int vidaEnemigo)`, que devuelve cuántos golpes necesita el héroe para dejar al enemigo en 0 o menos.

Datos del equipo: vida = _____ ataque = _____ golpes contra 100 = _____

☐ Coincide con la cuenta hecha a mano por el Cronista, contra 100 y contra 250 de vida.

☐ Usa `return`. Pista: un `while` que reste el ataque y cuente vueltas lo resuelve sin fórmulas.

20 XP

Reto 1.1

Mascota digital

Clase `Mascota` con `nombre` y `energia`. `jugar()` resta 15; `comer(int porcion)` suma; `String estado()` devuelve "Feliz" con 50 o más y "Cansada" si no. Prueben: energía 60, jugar dos veces, comer 20.

☐ `estado()` usa `return` y la salida es "Feliz".

20 XP

Reto 1.3

El clon honesto

Método `Heroe clonar()`: crea con `new` un héroe con los mismos datos y el nombre terminado en " (clon)". El clon recibe 50 de daño; impriman la vida de ambos.

☐ El original conserva su vida y lo muestran con un mapa de memoria.

Cazabugs

Main.java (tiene 2 bugs)

```
1 public class Main {
2     public static void main(String[] args) {
3         Heroe luna;
4         luna.nombre = "Luna";
5         System.out.println(luna.saludar());
6     }
7 }
8
9 class Heroe {
10     String nombre;
11     void saludar() {
12         return "Hola, soy " + nombre;
13     }
14 }
```

Encuentra y corrige

Debería imprimir "Hola, soy Luna", pero no compila. Escriban línea y corrección. Cada bug vale +10 XP. Ojo: algunos errores solo aparecen al corregir el anterior.

1

2

Checkpoint del Nivel 1

Marca con honestidad	Lo domino	Casi	Ayuda
Explico con un ejemplo propio la diferencia entre una clase y un objeto.	☐	☐	☐
Creo objetos con <code>new</code> y uso sus atributos y métodos con el punto.	☐	☐	☐
Sé cuándo un método lleva <code>void</code> y cuándo lleva un tipo con <code>return</code> .	☐	☐	☐

XP del nivel

Firma del docente: bonus sin Oráculo

Firma: explicación oral

La bóveda del gremio

constructor this private public get set

Misión

Alguien en la arena escribió `kira.vida = 9999;` y se volvió invencible. Otro jugador creó un héroe sin datos que deambula como `null`. El gremio blindó sus moldes: todo héroe nacerá completo gracias a un constructor, y nadie tocará su vida sin pasar por la puerta que la vigila.

Al terminar este nivel podrás

- ✓ Escribir constructores que dejan el objeto completo desde que nace.
- ✓ Usar `this` para distinguir atributo y parámetro, y `this(...)` para reutilizar constructores.
- ✓ Aplicar los cuatro niveles de visibilidad y leer sus símbolos en UML.
- ✓ Proteger atributos con `private` y validar cambios con getters y setters.

Botín de XP

3 predicciones relámpago	30
Retos 2.1, 2.2 y 2.3	90
Cazabugs (3 bugs)	30
Total del nivel	150
Bonus sin Oráculo y explicación oral	+40

Reloj del nivel

7' Modelado	5' Guiada	3' Predice	8' Retos	2' Bugs
----------------	--------------	---------------	-------------	------------

Ejemplo base 3: el constructor

Nivel 2

Un **constructor** es el ritual de nacimiento de un objeto: se ejecuta solo, una vez, en el instante del `new`. Sirve para que ningún héroe nazca incompleto.

► Ejemplo base 3. Dos constructores

```
1 public class Main {
2     public static void main(String[] args) {
3         Heroe kira = new Heroe("Kira", 100, 12);
4         Heroe pip = new Heroe("Pip");
5
6         kira.saludar();
7         pip.saludar();
8     }
9 }
10
11 class Heroe {
12     String nombre;
13     int vida;
14     int ataque;
15
16     Heroe(String nombre, int vida, int ataque) {
17         this.nombre = nombre;
18         this.vida = vida;
19         this.ataque = ataque;
20     }
21
22     Heroe(String nombre) {
23         this(nombre, 100, 10);
24     }
25
26     void saludar() {
27         System.out.println(nombre + ": vida " + vida
28             + ", ataque " + ataque);
29     }
30 }
```

Salida en consola

```
Kira: vida 100, ataque 12
Pip: vida 100, ataque 10
```

Cómo funciona, línea por línea

- L16** Se llama **igual que la clase** y no tiene tipo de retorno, ni siquiera `void`. Java lo ejecuta automáticamente con `new`.
- L17** `this.nombre` es el atributo del objeto; `nombre` a secas es el parámetro. Sin `this`, Java usaría el parámetro en ambos lados.
- L22** Un segundo constructor con otra lista de parámetros. Tener varios se llama **sobrecarga de constructores**.
- L23** `this(...)` llama al otro constructor y completa los valores que faltan. Debe ser la primera instrucción.
- L3-4** Java elige el constructor según cuántos datos se envían y de qué tipo.

Reglas del constructor

Regla	Si la rompes...
Mismo nombre que la clase	No compila: Java lo toma como un método sin tipo
Sin tipo de retorno, ni <code>void</code>	Se vuelve un método común y deja de ser constructor
Si escribes uno, Java ya no regala el vacío	<code>new Heroe()</code> deja de compilar
<code>this(...)</code> va primero	Error de compilación



¿Recuerdan el fantasma del Nivel 1 que decía Soy `null`? Con este constructor ya no puede existir: para crear un héroe hay que darle nombre.

Ejemplo base 4: la bóveda

Nivel 2

Encapsular es guardar los datos en una bóveda y abrir solo las puertas necesarias. Los atributos van `private`; los métodos que otros usan van `public`.

▶ Ejemplo base 4. Encapsulamiento

```
Main.java
1 public class Main {
2     public static void main(String[] args) {
3         Heroe kira = new Heroe("Kira", 9999);
4         System.out.println(kira.getVida());
5
6         kira.setVida(-40);
7         System.out.println(kira.getVida());
8
9         // kira.vida = 9999;
10    }
11 }
12
13 class Heroe {
14     private String nombre;
15     private int vida;
16
17     public Heroe(String nombre, int vida) {
18         this.nombre = nombre;
19         setVida(vida);
20     }
21
22     public String getNombre() {
23         return nombre;
24     }
25
26     public int getVida() {
27         return vida;
28     }
29
30     public void setVida(int vida) {
31         if (vida < 0) {
32             this.vida = 0;
33         } else if (vida > 150) {
34             this.vida = 150;
35         } else {
36             this.vida = vida;
37         }
38     }
39 }
40
41 Salida en consola
42 150
43 0
```

Cómo funciona

L14-15 `private`: solo el código escrito dentro de `Heroe` puede leer o cambiar estos atributos. Para `main`, `kira.vida` está bajo llave.

L19 El constructor también pasa por el setter, así la regla de 0 a 150 se cumple desde el nacimiento.

L22, L26 **Getters**: puertas de solo lectura. Dejan mirar sin tocar.

L30 **Setter**: puerta de escritura con guardia. Decide si acepta, corrige o rechaza el valor.

L3-6 El 9999 se convierte en 150 y el -40 en 0. La clase se defiende sola: nadie la engaña desde afuera. Eso es **encapsulamiento**.



Regla práctica del gremio

Atributos `private`, métodos útiles `public` y setters solo cuando cambiar el dato tenga sentido. Como en este taller todo vive en `Main.java`, todas las clases están en el mismo paquete: `protected` y “sin modificador” se comportan igual aquí, pero no en un proyecto con paquetes.

Los cuatro niveles de visibilidad

Modificador	UML	Misma clase	Mismo paquete	Subclase en otro paquete	Cualquier clase
<code>private</code>	-	sí	no	no	no
sin modificador	~	sí	sí	no	no
<code>protected</code>	#	sí	sí	sí	no
<code>public</code>	+	sí	sí	sí	sí

Usen el ejemplo base 4 completo. El Cronista escribe antes de que el Piloto ejecute.

- 1**
Predice
Si quitan las `//` de la línea `kira.vida = 9999;`, ¿qué pasa? Encierran una: **(a)** imprime 9999 **(b)** se ajusta a 150 **(c)** no compila.
- 2**
Ejecuta
Háganlo y copien el mensaje del compilador:
- 3**
Investiga
Vuelvan a comentar esa línea. Agreguen en `main`: `Heroe nadie = new Heroe();` ¿Por qué ya no compila, si en el Nivel 1 sí funcionaba?
- 4**
Modifica
Agreguen a la clase este método y prueben `kira.setVida(80);` seguido de `kira.recibirDanio(200);`. ¿Qué vida queda? ¿Por qué nunca será negativa?

```
método nuevo
1 public void recibirDanio(int danio) {
2     setVida(vida - danio);
3 }
```
- 5**
Crea
Pasen a los retos: ahora ustedes diseñan qué se protege y qué puertas se abren.

Predicciones relámpago

Sin ejecutar. Tres minutos para las tres.

Predicción A +10	Predicción B +10	Predicción C +10
<pre>1 class Cofre { 2 int monedas; 3 4 Cofre() { 5 this(50); 6 } 7 8 Cofre(int monedas) { 9 this.monedas = monedas * 2; 10 } 11 } 12 13 // en main 14 Cofre c = new Cofre(); 15 System.out.println(c.monedas);</pre>	<pre>1 class Gema { 2 int valor; 3 4 Gema(int valor) { 5 valor = valor; 6 } 7 } 8 9 // en main 10 Gema g = new Gema(30); 11 System.out.println(g.valor);</pre>	<pre>1 // usa el Heroe del ejemplo base 4 2 Heroe rin = new Heroe("Rin", 80); 3 rin.setVida(rin.getVida() + 100); 4 System.out.println(rin.getVida());</pre>
Imprime: _____	Imprime: _____	Imprime: _____



Si la B los sorprendió: la sombra del parámetro

En la predicción B, dentro del constructor hay dos cosas llamadas `valor`: el parámetro y el atributo. El parámetro, más cercano, “tapa” al atributo como una sombra. Por eso el truco del gremio es escribir siempre `this.` al asignar atributos.



Reto 2.1

La bóveda del gremio

Crean `CuentaGremio` con `titular` y `saldo` privados. El constructor recibe solo el titular (el nombre de su gremio) y el saldo arranca en 0. Métodos: `depositar(int monto)` acepta solo montos positivos; `boolean retirar(int monto)` retira solo si alcanza y devuelve si pudo; `getSaldo()`. Ejecuten en orden: depositar 500, depositar -100, retirar 800, retirar 200. Impriman los dos `boolean` y el saldo.

- ☐ La salida es false, true y saldo 300.
- ☐ No existe `setSaldo`. Explican al docente por qué sería un agujero en la bóveda.



Reto 2.2

Contraseña blindada

Clase `Usuario` con `nombre` y `clave` privados. Métodos `boolean verificar(intento)` y `boolean cambiarClave(actual, nueva)`: cambia solo si la actual es correcta y la nueva tiene 8 caracteres o más. Usuario: el Cronista. Clave inicial: nombre del gremio en minúsculas más el número de integrantes.

- ☐ No hay `getClave()`.
- ☐ Probaron actual incorrecta, nueva corta y cambio válido.

💡 Los textos se comparan con `.equals()`, no con `==`. Y `texto.length()` da su tamaño.



Reto 2.3

Tres formas de nacer

Clase `Pocion` con `nombre` y `curacion` privados y tres constructores: `Pocion()` crea “Poción básica” que cura 20; `Pocion(int curacion)` crea “Poción personalizada”; `Pocion(String nombre, int curacion)` asigna los atributos. Método `String describir()`.

- ☐ Solo el tercer constructor `this(...)` asigna atributos; los otros dos usan
- ☐ Crean e imprimen una poción de cada tipo.

Cazabugs

●●● Main.java (tiene 3 bugs)

```
1 public class Main {
2     public static void main(String[] args) {
3         Arma espada = new Arma("Espada", 25);
4         System.out.println(espada.getNombre() + ": " + espada.getDanio());
5     }
6 }
7
8 class Arma {
9     private String nombre;
10    private int danio;
11
12    public void Arma(String nombre, int danio) {
13        nombre = nombre;
14        this.danio = danio;
15    }
16
17    public String getNombre() {
18        return nombre;
19    }
20
21    private int getDanio() {
22        return danio;
23    }
24 }
```

Encuentra y corrige

Debería imprimir "Espada: 25". Tiene dos errores de compilación y uno de lógica que solo se ve cuando compila. Escriban línea y corrección. Cada bug vale **+10 XP**.

1

2

3

Si activan el Oráculo aquí, no le pidan “corrige este código”. Pregúntenle: “¿Qué diferencia hay entre un constructor y un método que se llama igual que la clase?”. Los bugs los encuentran ustedes.

Checkpoint del Nivel 2

Marca con honestidad	Lo domino	Casi	Ayuda
Escribo un constructor y sé por qué no lleva <code>void</code> .	☐	☐	☐
Uso <code>this</code> para diferenciar atributo y parámetro.	☐	☐	☐
Explico qué protege <code>private</code> y para qué sirven getters y setters.	☐	☐	☐
Leo los símbolos <code>+</code> <code>-</code> <code>#</code> <code>~</code> de un diagrama UML.	☐	☐	☐

XP del nivel

Firma del docente: bonus sin Oráculo

Firma: explicación oral

Pausa activa: guardar partida

10 min

✂ Polimorfismo humano (6 min)

Todo el salón se pone de pie y se vuelve un arreglo de personajes según su mes de cumpleaños: enero a abril, Guerrero; mayo a agosto, Mago; septiembre a diciembre, Arquero. El docente proyecta este pseudocódigo y el grupo lo ejecuta con el cuerpo. Quien se equivoca se sienta... y vuelve a entrar en la siguiente instrucción.

```
ejecútalo con el cuerpo

1 Personaje[] salon = todosDePie();
2
3 for (Personaje p : salon) {
4     p.saludar();    // heredado
5 }
6 for (Personaje p : salon) {
7     p.atacar();    // sobrescrito
8 }
9 for (Personaje p : salon) {
10     if (p instanceof Mago) {
11         ((Mago) p).levitar();
12     }
13 }
14 docente.silbar();    // final
```

Clase	saludar()	atacar()
Guerrero	Mano arriba	Puño al aire y "¡Ja!"
Mago	Mano arriba	Manos al frente y "¡Fuego!"
Arquero	Mano arriba	Tensa un arco y "¡Fiu!"



Si alguien que no es Mago se pone a levitar, acaba de lanzar un `ClassCastException`: se sienta. Si el docente "olvida" el `if` y todos levitan, el salón entero grita "¡no compila!".

📁 Guardar partida (4 min)

- ✓ Copien el código de los niveles 1 y 2 en el documento del equipo.
- ✓ El docente actualiza la tabla de clasificación y anuncia los tres primeros.
- ✓ Agua, estiramiento y roles listos para rotar en el Nivel 3.

El linaje

`extends` `super` `protected` `@Override`

Misión

El gremio ya tiene guerreros y magos, pero cada clase repetía nombre, vida, ataque y presentarse(): decenas de líneas copiadas y los mismos bugs en dos lugares. Los forjadores crean un ancestro común, Personaje, del que todos heredan. Se escribe una vez y lo reciben todos.



Al terminar este nivel podrás

- ✓ Crear subclases con `extends` y reconocer qué se hereda y qué no.
- ✓ Llamar al constructor del padre con `super(...)`.
- ✓ Sobrescribir un método con `@Override` y reutilizar la versión del padre.
- ✓ Dibujar la jerarquía en UML y aplicar la prueba del “es un”.

Botín de XP

3 predicciones y rompecabezas	40
Retos 3.1, 3.2 y 3.3	80
Cazabugs (3 bugs)	30
Total del nivel	150
Bonus sin Oráculo y explicación oral	+40

Reloj del nivel

7' Modelado	5' Guiada	4' Predice	7' Retos	2' Bugs
----------------	--------------	---------------	-------------	------------

🔄 **Rota los roles ahora.** Cronista pasa a Auditor de IA.

Ejemplo base 5: la herencia

Nivel 3

Con `extends`, una clase **hija** recibe los atributos y métodos de su clase **padre** y agrega lo suyo. Guerrero y Mago **son** personajes.

▷ Ejemplo base 5. Padre y main

```
1 public class Main {
2     public static void main(String[] args) {
3         Guerrero brako = new Guerrero("Brako", 120, 15, 8);
4         Mago lira = new Mago("Lira", 70, 10, 50);
5
6         brako.presentarse();
7         brako.defender();
8         lira.presentarse();
9         lira.lanzarHechizo();
10    }
11 }
12
13 class Personaje {
14     protected String nombre;
15     protected int vida;
16     protected int ataque;
17
18     public Personaje(String nombre, int vida,
19                     int ataque) {
20         this.nombre = nombre;
21         this.vida = vida;
22         this.ataque = ataque;
23     }
24
25     public void presentarse() {
26         System.out.println(nombre + " | vida " + vida
27                             + " | ataque " + ataque);
28     }
29 }
```

🔗 Las hijas, debajo

```
31 class Guerrero extends Personaje {
32     private int armadura;
33
34     public Guerrero(String nombre, int vida,
35                     int ataque, int armadura) {
36         super(nombre, vida, ataque);
37         this.armadura = armadura;
38     }
39
40     public void defender() {
41         System.out.println(nombre + " se defiende: +"
42                             + armadura);
43     }
44 }
45
46 class Mago extends Personaje {
47     private int mana;
48
49     public Mago(String nombre, int vida,
50                 int ataque, int mana) {
51         super(nombre, vida, ataque);
52         this.mana = mana;
53     }
54
55     public void lanzarHechizo() {
56         mana = mana - 20;
57         System.out.println(nombre + " hechiza. Maná: "
58                             + mana);
59     }
60 }
```

Salida en consola

```
Brako | vida 120 | ataque 15
Brako se defiende: +8
Lira | vida 70 | ataque 10
Lira hechiza. Maná: 30
```

Cómo funciona, línea por línea

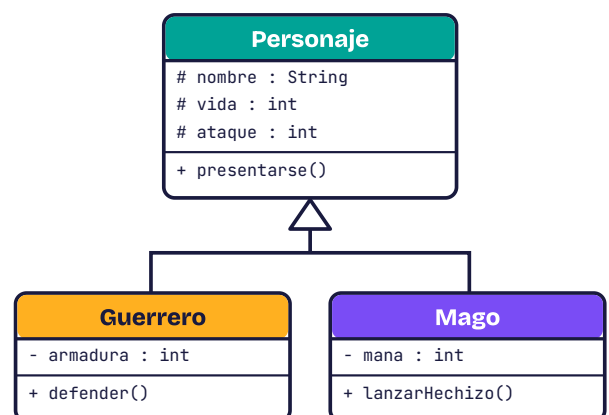
L14 `protected`: privado para el mundo, pero visible para las clases hijas. Así Guerrero puede usar `nombre`.

L31 `extends Personaje`: Guerrero hereda `nombre`, `vida`, `ataque` y `presentarse()` sin reescribirlos.

L36 `super(...)` llama al constructor del padre para que llene su parte. Va en la primera línea; los constructores **no** se heredan.

L37 Luego la hija llena lo que es solo suyo.

L6 `presentarse()` no está escrito en Guerrero, pero funciona: lo heredó.



En UML, la herencia es una flecha con triángulo vacío que apunta al padre.

Ejemplo base 6: sobrescribir sin copiar

Nivel 3

Una hija puede **redefinir** un método heredado para hacerlo a su manera. Si además quiere conservar lo que hacía el padre, lo llama con `super.`. Agreguen este método dentro de `Guerrero`, en el ejemplo base 5:

Ejemplo base 6. @Override

```
dentro de class Guerrero
1 @Override
2 public void presentarse() {
3     super.presentarse();
4     System.out.println("    Guerrero con "
5         + armadura + " de armadura");
6 }
```

Salida en consola

```
Brako | vida 120 | ataque 15
    Guerrero con 8 de armadura
Brako se defiende: +8
Lira | vida 70 | ataque 10
Lira hechiza. Maná: 30
```

Cómo funciona

- L1 Anotación que le pide al compilador: “verifica que esto sí sobrescribe algo del padre”. Si hay un error de nombre, lo detecta.
- L2 Misma firma que en Personaje: mismo nombre, mismos parámetros, mismo tipo de retorno.
- L3 Ejecuta primero la versión del padre, sin copiar su código.
- L4 Después agrega lo propio. Mago sigue usando la versión original, porque no la sobrescribió.

¿Qué recibe una hija?

Elemento del padre	¿Lo usa la hija?
Atributos y métodos <code>public</code> o <code>protected</code>	Sí, directamente
Atributos <code>private</code>	Existen, pero solo vía getters
Constructores	No: se llaman con <code>super(...)</code>
Métodos que quiera cambiar	Sí, con <code>@Override</code>



Java permite **un solo padre** por clase: `extends` admite un único nombre. Y toda clase hereda en el fondo de `Object`.

La prueba del “es un”

Hereda solo si la frase suena verdadera. Marquen sí o no en 30 segundos.

Frase	Sí	No
Un Guerrero es un Personaje		
Una Espada es un Personaje		
Un Mago es un Guerrero		
Un Escuadrón es un Personaje		
Un Dragón es un Personaje		

Si la frase correcta es “**tiene un**”, no es herencia: es agregación, y la verán en el Nivel 5.

Práctica guiada

- 1

Predice y ejecuta

En el ejemplo base 5, ¿qué pasa si borran `super(nombre, vida, ataque);` del constructor de Guerrero? Escriban la predicción, háganlo y copien el error: _____
- 2

Investiga

Restauren `super`. Cambien `protected String nombre;` por `private String nombre;`. ¿Qué líneas fallan ahora y por qué no falla `presentarse()`? _____
- 3

Modifica

Vuelvan a `protected`. Sobrescriban `presentarse()` en Mago con `super.presentarse()` y una línea que muestre el maná.

Predicciones y rompecabezas

40 XP

Sin ejecutar. Cuatro minutos para todo. Luego lo comprueban en el compilador.

Predicción A +10	Predicción B +10	Predicción C +10
<pre>1 class A { 2 A() { System.out.println("A"); } 3 } 4 class B extends A { 5 B() { System.out.println("B"); } 6 } 7 class C extends B { 8 C() { System.out.println("C"); } 9 } 10 11 // en main 12 new C();</pre>	<pre>1 class Animal { 2 String sonido() { return "..."; } 3 } 4 class Gato extends Animal { 5 @Override 6 String sonido() { 7 return "Miau" + super.sonido(); 8 } 9 } 10 11 // en main 12 Gato g = new Gato(); 13 System.out.println(g.sonido());</pre>	<pre>1 // usa el ejemplo base 5 2 Mago lira = 3 new Mago("Lira", 70, 10, 50); 4 lira.presentarse(); 5 lira.defender(); 6 lira.lanzarHechizo();</pre>
Imprime: _____	Imprime: _____	¿Qué líneas NO compilan? _____

Rompecabezas del linaje +10

Estas piezas forman la clase `Sanador`, hija de `Personaje`, con su constructor y un `presentarse()` sobrescrito. Numeren del 1 al 9 en el orden correcto. **Dos piezas son trampas:** táchenlas. Las llaves de cierre se sobreentienden.

1	class Sanador extends Personaje {
2	public void Sanador(String n, int v, int a, int hierbas) {
3	super.presentarse();
4	private int hierbas;
5	@Override
6	System.out.println(" Sanador con " + hierbas);
7	this.hierbas = hierbas;
8	public Sanador(String n, int v, int a, int hierbas) {
9	class Sanador implements Personaje {
10	super(n, v, a);
11	public void presentarse() {



¿Cómo detectar las trampas?

Una trampa convierte al constructor en un método común; la otra usa una palabra que en Java sirve para interfaces, no para clases. Cuando terminen, escríbanlo y ejecútenlo con `new Sanador("Mira", 80, 6, 3).presentarse();`

Reto 3.1

Nace el Arquero

Crean `Arquero` extends `Personaje` con `private int flechas`. Su constructor recibe cuatro datos y llama a `super`. Método `disparar()`: si quedan flechas, resta una e imprime cuántas quedan; si no, avisa que no tiene. Prueben con 2 flechas y 3 disparos.

- ☐ `super(...)` es la primera línea del constructor.
- ☐ El tercer disparo avisa que no hay flechas.

20
XP

Reto 3.3

Grito de guerra

Sobrescriban `presentarse()` en Guerrero, Mago y Arquero. Cada versión llama primero a `super.presentarse()` y luego imprime un grito de guerra inventado por el equipo con el nombre del gremio y el atributo propio de esa clase.

- ☐ `@Override` en las tres clases.
- ☐ Ninguna hija repite el `println` del padre.

30
XP

Reto 3.2

El árbol del gremio en UML

En papel y sin pantalla: dibujen el diagrama de clases de `Personaje`, `Guerrero`, `Mago` y `Arquero` (el del reto 3.1) con atributos, métodos, símbolos `+` `-` `#` y flechas de herencia con triángulo vacío. El docente lo revisa en 60 segundos.

- ☐ Símbolos de visibilidad correctos.
- ☐ Las hijas no repiten lo que ya está en `Personaje`.
- ☐ Cada flecha termina en triángulo vacío apuntando al padre.

30
XP

Cazabugs

● ● ● Main.java (tiene 3 bugs)

```
1 public class Main {
2     public static void main(String[] args) {
3         Grifo g = new Grifo(40);
4         g.mostrar();
5     }
6 }
7
8 class Montura {
9     private int velocidad;
10
11     public Montura(int velocidad) {
12         this.velocidad = velocidad;
13     }
14
15     public void mostrar() {
16         System.out.println("Velocidad: " + velocidad);
17     }
18 }
19
20 class Grifo extends Montura {
21     public Grifo(int velocidad) {
22         System.out.println("Nace un grifo");
23     }
24
25     @Override
26     public void mostrar() {
27         velocidad = velocidad + 10;
28         super.mostrar();
29     }
30 }
```

 **Encuentra y corrige**

Debería imprimir "Nace un grifo" y luego "Velocidad: 50". No compila por tres razones distintas. Escriban línea y corrección. Cada bug vale **+10 XP**.

1

2

3

1=

2=

Estrategia del cazador

Cuando el compilador muestra varios errores, corrijan **primero el de número de línea más bajo** y vuelvan a compilar. Un solo error puede provocar otros en cascada, y a veces uno nuevo aparece solo cuando desaparece el anterior.



Si activan el Oráculo, pregunten “¿qué significa *method does not override or implement a method from a supertype?*” y no “arregla mi código”. La corrección la escriben ustedes.

Checkpoint del Nivel 3

Marca con honestidad	Lo domino	Casi	Ayuda
Explico qué se hereda con <code>extends</code> y qué no.	☐	☐	☐
Sé por qué <code>super(...)</code> aparece en el constructor de la hija.	☐	☐	☐
Sobrescribo con <code>@Override</code> y reutilizo con <code>super.metodo()</code> .	☐	☐	☐
Aplico la prueba del “es un” antes de decidir heredar.	☐	☐	☐

XP del nivel

Firma del docente: bonus sin Oráculo

Firma: explicación oral

La arena de los mil rostros

```
Personaje p = new Mago(...) @Override instanceof
```

Misión

El árbitro de la arena tiene una sola lista de combatientes y una sola orden: “¡ataquen!”. No pregunta quién es guerrero y quién es mago: cada uno responde a su manera. Eso es polimorfismo, una misma llamada con muchas formas. Tu misión es montar un combate donde sumar un nuevo tipo de personaje no obligue a tocar el ciclo.

Al terminar este nivel podrás

- ✓ Guardar objetos de distintas subclases en un arreglo del tipo padre.
- ✓ Predecir qué versión de un método se ejecuta según el objeto real.
- ✓ Distinguir sobrecarga de sobrescritura en código propio.
- ✓ Usar `instanceof` y el casting sin provocar `ClassCastException`.

Botín de XP

3 predicciones relámpago	30
Retos 4.1, 4.2 y 4.3	90
Cazabugs (3 bugs)	30
Total del nivel	150
Bonus sin Oráculo y explicación oral	+40

Reloj del nivel

7' Modelado	5' Guiada	3' Predice	8' Retos	2' Bugs
-----------------------	---------------------	----------------------	--------------------	-------------------

Ejemplo base 7: una orden, muchas respuestas

Nivel 4

Polimorfismo: una variable del tipo padre puede apuntar a cualquier hija, y al llamar un método se ejecuta la versión del **objeto real**, no la de la etiqueta.

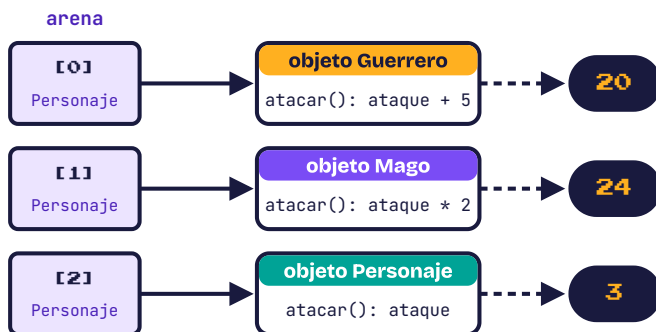
► Ejemplo base 7. La arena

📁 Clases, debajo de Main

```
1 public class Main {
2     public static void main(String[] args) {
3         Personaje[] arena = {
4             new Guerrero("Brako", 120, 15),
5             new Mago("Lira", 70, 12),
6             new Personaje("Aldeano", 40, 3)
7         };
8
9         int total = 0;
10        for (Personaje p : arena) {
11            int danio = p.atacar();
12            System.out.println(p.getNombre() + " hace "
13                + danio + " de daño");
14            total = total + danio;
15        }
16        System.out.println("Daño total: " + total);
17    }
18 }
```

Salida en consola

```
Brako hace 20 de daño
Lira hace 24 de daño
Aldeano hace 3 de daño
Daño total: 47
```



Tres cajas con la misma etiqueta, tres objetos distintos. La flecha punteada es lo que devuelve `atacar()` en cada vuelta.

```
20 class Personaje {
21     protected String nombre;
22     protected int vida;
23     protected int ataque;
24
25     public Personaje(String nombre, int vida,
26         int ataque) {
27         this.nombre = nombre;
28         this.vida = vida;
29         this.ataque = ataque;
30     }
31
32     public String getNombre() { return nombre; }
33
34     public int atacar() {
35         return ataque;
36     }
37 }
38
39 class Guerrero extends Personaje {
40     public Guerrero(String nombre, int vida, int ataque) {
41         super(nombre, vida, ataque);
42     }
43
44     @Override
45     public int atacar() {
46         return ataque + 5;
47     }
48 }
49
50 class Mago extends Personaje {
51     public Mago(String nombre, int vida, int ataque) {
52         super(nombre, vida, ataque);
53     }
54
55     @Override
56     public int atacar() {
57         return ataque * 2;
58     }
59
60     public void lanzarHechizo() {
61         System.out.println(nombre + " lanza un rayo");
62     }
63 }
```

Cómo funciona, línea por línea

L3 Un arreglo de `Personaje`. Como Guerrero y Mago son personajes, caben ahí. Esto se llama *upcasting* y es automático.

L10 En cada vuelta, `p` apunta a un objeto distinto. El ciclo no sabe ni le importa de qué clase es.

L11 Java mira el objeto real en tiempo de ejecución y llama su versión de `atacar()`: 20, 24 y 3. Esto se llama **enlace dinámico**.

L46 Si mañana nace un Arquero con su propio `atacar()`, el `for` funciona sin cambiar una sola línea.

Ejemplo base 8: la etiqueta y el objeto

Nivel 4

► Ejemplo base 8. Dentro de main

```
main (con las clases del ejemplo 7)

1 Personaje p = new Mago("Lira", 70, 12);
2 System.out.println(p.atacar());
3 // p.lanzarHechizo();
4
5 if (p instanceof Mago) {
6     Mago m = (Mago) p;
7     m.lanzarHechizo();
8 }
9
10 Guerrero brako = new Guerrero("Brako", 120, 15);
11 System.out.println(brako.atacar());
12 System.out.println(brako.atacar(10));
```

Salida en consola

```
24
Lira lanza un rayo
20
30
```

+ Agreguen esto dentro de Guerrero

```
sobrecarga en Guerrero

1 public int atacar(int furia) {
2     return atacar() + furia;
3 }
```

Cómo funciona

- L1** La etiqueta es `Personaje`; el objeto real es un `Mago`.
- L2** Imprime 24: se ejecuta la versión del `Mago`, porque manda el objeto.
- L3** Si quitan el comentario, no compila: la etiqueta `Personaje` no conoce `lanzarHechizo()`.
- L5** `instanceof` pregunta "¿este objeto es un `Mago`?". Solo si lo es, el cast `(Mago) p` crea una etiqueta más específica.
- L12** Otro `atacar`, pero con un parámetro: es **sobrecarga**. Java elige cuál usar por los datos que envías.



Las dos preguntas del polimorfismo

- ¿Puedo llamarlo? Lo decide la etiqueta, al compilar.
- ¿Qué versión corre? Lo decide el objeto, al ejecutar.

Sobrescritura o sobrecarga

Aspecto	Sobrescritura (<i>override</i>)	Sobrecarga (<i>overload</i>)
Dónde	En la clase hija, sobre un método del padre	En la misma clase (o en la hija), con el mismo nombre
Parámetros	Exactamente los mismos	Distintos en cantidad o en tipo
Quién decide	El objeto real, al ejecutar	Los argumentos escritos, al compilar
Ejemplo	<code>atacar()</code> de Guerrero reemplaza al de Personaje	<code>atacar()</code> y <code>atacar(int furia)</code> conviven

Práctica guiada

1

Si agregan `new Mago("Nox", 60, 20)` al arreglo del ejemplo base 7, ¿cuál será el daño total? _____

Predice

2

Ejecuten para comprobar. Luego borren el método `atacar()` de Guerrero, con su `@Override`. ¿Cuánto hace Brako ahora y por qué? _____

Investiga

3

Restauran el método. Dentro del `for`, hagan que solo los magos lancen su hechizo usando `instanceof` y un cast. Sin el `if`, ¿por qué no compila? _____

Modifica

Predicciones y retos del Nivel 4

120 XP

Predicciones sin ejecutar, con las clases del ejemplo base 7. Tres minutos.

Predicción A

+10

```
1 Personaje p =  
2 new Guerrero("Brako", 120, 15);  
3 System.out.println(p.atacar());
```

Imprime:

Predicción B

+10

```
1 Personaje[] eq = {  
2 new Mago("A", 50, 4),  
3 new Mago("B", 50, 4),  
4 new Guerrero("C", 50, 4)  
5 };  
6 int t = 0;  
7 for (Personaje x : eq) {  
8 t = t + x.atacar();  
9 }  
10 System.out.println(t);
```

Imprime:

Predicción C

+10

```
1 class Dado {  
2 String tirar() {  
3 return "d6";  
4 }  
5 String tirar(int caras) {  
6 return "d" + caras;  
7 }  
8 }  
9  
10 // en main  
11 Dado d = new Dado();  
12 System.out.println(d.tirar());  
13 System.out.println(d.tirar(20));
```

Imprime (2 líneas):



30
XP

Reto 4.1

El Arquero entra a la arena

Agreguen un `Arquero` con `flechas` que sobrescribe `atacar()`: si le quedan flechas, gasta una y devuelve `ataque * 3`; si no, devuelve 1. Metan `new Arquero(nombre, 90, 8, 1)` en la arena, con el nombre del Navegante, y envuelvan el `for` en otro que haga dos rondas e imprima el daño total de cada una.

- ☐ El ciclo interno no pregunta de qué clase es cada combatiente.
- ☐ Las dos rondas dan totales distintos y el equipo explica por qué.

Reto 4.2

Figuras de tu mesa

Con una regla, midan en centímetros algo circular, algo rectangular y algo triangular de su mesa. Clase `Figura` con `double area()` que devuelve 0, y tres hijas que la sobrescriben. Guárdenlas en un `Figura[]` y sumen las áreas en un solo `for`.

Radio _____ Base x altura _____
Triángulo _____

- ☐ Medidas reales anotadas antes de programar.
- ☐ Un solo ciclo recorre el arreglo.

💡 Círculo: `Math.PI * r * r`. Triángulo: base por altura entre 2.



30
XP

Reto 4.3

Furia sobrecargada

En `Guerrero`, sobrecarguen `atacar(int furia)`, que devuelve `atacar() + furia`. En `Mago`, `atacar(boolean critico)`: si es crítico, el doble de `atacar()`. Prueben `brako.atacar(10)` y `lira.atacar(true)`. Después escriban `Personaje p = brako;` y `p.atacar(10);`.

- ☐ Obtienen 30 y 48.
- ☐ Explican por qué `p.atacar(10)` no compila.

Cazabugs

Main.java (tiene 3 bugs)

```
1 class Mago extends Personaje {
2     public Mago(String nombre, int vida, int ataque) {
3         super(nombre, vida, ataque);
4     }
5
6     @Override
7     public int atacar(int bonus) {
8         return ataque * 2 + bonus;
9     }
10
11 // dentro de main (Personaje y Guerrero del ejemplo 7)
12 Personaje[] arena = new Personaje[3];
13 arena[0] = new Mago("Lira", 70, 12);
14 arena[1] = new Guerrero("Brako", 120, 15);
15
16 for (Personaje p : arena) {
17     System.out.println(p.getNombre() + ": " + p.atacar());
18 }
19
20 Mago elegido = (Mago) arena[1];
21 elegido.lanzarHechizo();
```

Encuentra y corrige

Debería mostrar el daño de tres combatientes y luego un hechizo. Hay un error de compilación y dos que explotan al ejecutar. Escriban línea y corrección. Cada bug vale **+10 XP**.

1

2

3



Errores que el compilador no ve

`NullPointerException`: intentaron usar una referencia que no apunta a nada. `ClassCastException`: intentaron poner una etiqueta que no corresponde al objeto real. Ambos compilan bien y explotan al ejecutar. El jefe final se llama Dragón Null por algo.

Checkpoint del Nivel 4

Marca con honestidad	Lo domino	Casi	Ayuda
Guardo objetos de distintas hijas en un arreglo del tipo padre.	☐	☐	☐
Predigo qué versión de un método se ejecuta mirando el objeto real.	☐	☐	☐
Distingo sobrecarga de sobrescritura con un ejemplo propio.	☐	☐	☐
Uso <code>instanceof</code> antes de un cast para evitar <code>ClassCastException</code> .	☐	☐	☐

XP del nivel

Firma del docente: bonus sin Oráculo

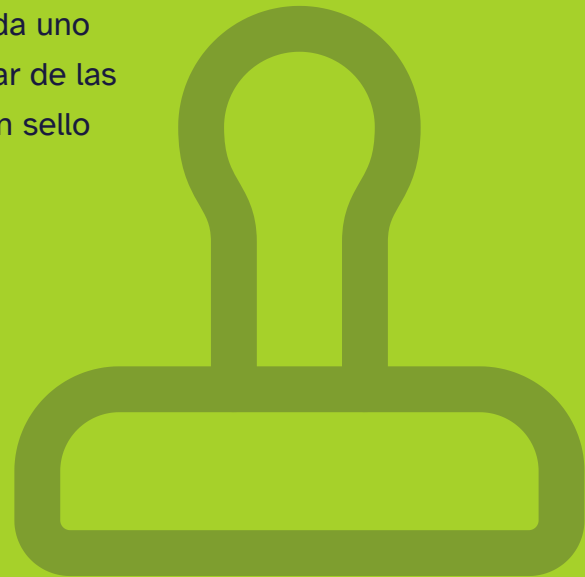
Firma: explicación oral

El escuadrón sellado

`ArrayList<Personaje>` tiene un `final static final`

Misión

Los héroes ya no pelean solos: forman escuadrones. Pero un escuadrón no es un héroe; **tiene** héroes, y si se disuelve, cada uno sigue su camino. Mientras tanto, un tramposo intentó heredar de las Reglas para subir la vida máxima. El gremio responde con un sello que ni la herencia puede romper: `final`.



Al terminar este nivel podrás

- ✓ Modelar una relación “tiene un” con agregación y dibujarla en UML.
- ✓ Distinguir herencia, agregación y composición con ejemplos propios.
- ✓ Usar `final` en atributos, métodos y clases, y explicar qué bloquea cada uno.
- ✓ Declarar constantes con `static final` y usarlas en validaciones.

Botín de XP

3 predicciones relámpago	30
Retos 5.1, 5.2 y 5.3	70
Cazabugs (2 bugs)	20
Total del nivel	120
Bonus sin Oráculo y explicación oral	+40

Reloj del nivel

6' Modelado	4' Guiada	2' Predice	6' Retos	2' Bugs
----------------	--------------	---------------	-------------	------------

🔄 **Rota los roles ahora.** Piloto pasa a Navegante.

Ejemplo base 9: el escuadrón

Nivel 5

Agregación: un objeto guarda referencias a otros objetos que existen por su cuenta.
El escuadrón no fabrica a sus héroes: los recibe.

▷ Ejemplo base 9. Main y Personaje

```
1 import java.util.ArrayList;
2
3 public class Main {
4     public static void main(String[] args) {
5         Personaje brako = new Personaje("Brako", 120);
6         Personaje lira = new Personaje("Lira", 70);
7
8         Escuadron alfa = new Escuadron("Alfa");
9         alfa.agregar(brako);
10        alfa.agregar(lira);
11        alfa.mostrar();
12
13        alfa = null;
14        System.out.println(brako.getNombre()
15            + " sigue con " + brako.getVida());
16    }
17 }
18
19 class Personaje {
20     private String nombre;
21     private int vida;
22
23     public Personaje(String nombre, int vida) {
24         this.nombre = nombre;
25         this.vida = vida;
26     }
27
28     public String getNombre() { return nombre; }
29     public int getVida() { return vida; }
30 }
```

🔗 La clase que agrega

```
32 class Escuadron {
33     private String nombre;
34     private ArrayList<Personaje> miembros =
35         new ArrayList<>();
36
37     public Escuadron(String nombre) {
38         this.nombre = nombre;
39     }
40
41     public void agregar(Personaje p) {
42         miembros.add(p);
43     }
44
45     public int vidaTotal() {
46         int total = 0;
47         for (Personaje p : miembros) {
48             total = total + p.getVida();
49         }
50         return total;
51     }
52
53     public void mostrar() {
54         System.out.println("Escuadrón " + nombre + ": "
55             + miembros.size() + " miembros, vida "
56             + vidaTotal());
57         for (Personaje p : miembros) {
58             System.out.println("  - " + p.getNombre());
59         }
60     }
61 }
```

Salida en consola

```
Escuadrón Alfa: 2 miembros, vida 190
- Brako
- Lira
Brako sigue con 120
```

Cómo funciona, línea por línea

L1 `ArrayList` es una lista que crece sola. Vive en `java.util`, por eso se importa arriba del todo.

L34 El escuadrón **tiene** una lista de referencias a personajes. `<Personaje>` indica qué tipo de objetos acepta.

L41 El personaje llega ya creado desde afuera. `add` guarda la referencia; no hay ningún `new Personaje` aquí.

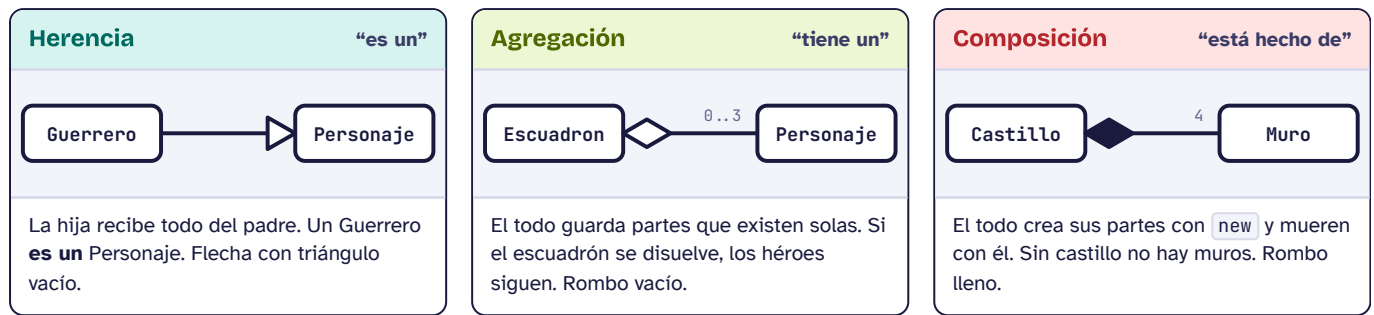
L45 El escuadrón recorre a sus miembros y trabaja con ellos a través de sus métodos públicos.

L13 La variable `alfa` deja de apuntar al escuadrón: se “disuelve”.

L15 Brako sigue vivo, porque la variable `brako` aún lo apunta. Las partes sobreviven al todo: eso distingue la agregación.

Tres formas de relacionar clases

Nivel 5



Ejemplo base 10: el sello final

Nivel 5

▷ Ejemplo base 10. Reglas selladas

🔒 Atributo y método sellados

```
1 public class Main {
2     public static void main(String[] args) {
3         Personaje kira = new Personaje("Kira", 200);
4         System.out.println(kira.getNombre() + ": "
5             + kira.getVida());
6         System.out.println("Máximo: "
7             + Reglas.VIDA_MAXIMA);
8     }
9 }
10
11 final class Reglas {
12     public static final int VIDAX_MAXIMA = 150;
13     public static final int MAX_MIEMBROS = 3;
14 }
```

Salida en consola

```
Kira: 150
Máximo: 150
```

```
16 class Personaje {
17     private final String nombre;
18     private int vida;
19
20     public Personaje(String nombre, int vida) {
21         this.nombre = nombre;
22         this.vida = Math.min(vida, Reglas.VIDA_MAXIMA);
23     }
24
25     public final String getNombre() {
26         return nombre;
27     }
28
29     public int getVida() {
30         return vida;
31     }
32 }
33
34 // class ReglasTramposas extends Reglas { }
```

final en...	Qué bloquea	La trampa y lo que responde el compilador
Constante	Asignarle otro valor	<code>Reglas.VIDA_MAXIMA = 9999;</code> cannot assign a value to static final variable VIDAX_MAXIMA
Atributo	Cambiarlo después del constructor	<code>this.nombre = "X";</code> dentro de un método cannot assign a value to final variable nombre
Método	Que una hija lo sobrescriba	<code>@Override public String getNombre()</code> en una hija getNombre() in Tramposo cannot override getNombre() in Personaje
Clase	Que alguien herede de ella	<code>class ReglasTramposas extends Reglas { }</code> cannot inherit from final Reglas



¿Y el static?

`static` hace que la constante sea de la clase y no de cada objeto: se usa como `Reglas.VIDA_MAXIMA` sin crear nada. Por convención, las constantes van en MAYÚSCULAS.



Dato curioso

`String`, `Math` e `Integer` son clases `final` de Java: nadie puede heredar de ellas y cambiar cómo funciona un texto o una suma.

Práctica guiada y predicciones

PRIMM

- 1** En el ejemplo base 9, justo después de `alfa = null;`, ¿qué imprime la última línea? ¿Brako desapareció?
Predice _____
- 2** Antes de `alfa = null;` creen `Escuadron beta` y agreguen a `brako` también. ¿Cuántos objetos Brako existen ahora? _____ ¿Qué cambiaría si `agregar` hiciera `new Personaje(...)` por dentro?
Investiga _____
- 3** Conviertan `agregar` en `boolean`: si ya hay `Reglas.MAX_MIEMBROS` miembros, devuelve `false` sin agregar. Agreguen cuatro personajes e impriman cada resultado. Esperan: _____
Modifica _____

Predicciones relámpago

Predicción A +10	Predicción B +10	Predicción C +10
<pre>1 // Personaje con recibirDanio(int) 2 Personaje k = 3 new Personaje("Kira", 100); 4 Escuadron a = new Escuadron("A"); 5 Escuadron b = new Escuadron("B"); 6 a.agregar(k); 7 b.agregar(k); 8 k.recibirDanio(40); 9 int total = a.vidaTotal() 10 + b.vidaTotal(); 11 System.out.println(total);</pre>	<pre>1 final int monedas = 10; 2 monedas = monedas + 5; 3 System.out.println(monedas);</pre>	<pre>1 final ArrayList<String> botin = 2 new ArrayList<>(); 3 botin.add("gema"); 4 botin.add("oro"); 5 System.out.println(botin.size());</pre>
Imprime: _____	Imprime o ¿no compila? _____	Imprime: _____



Si la C los confundió

Una referencia `final` no puede apuntar a **otro** objeto, pero el objeto al que apunta sí puede cambiar por dentro. Sellaron la flecha, no la caja.

Retos del Nivel 5

Reto 5.1

La mochila del Cronista

Pongan sobre la mesa cuatro objetos reales del Cronista y estimen su peso en gramos. Clase `Objeto` con `nombre` y `gramos` como atributos `final`. Clase `Mochila` con `private final int capacidad` de 1000 g, una lista de objetos, `int pesoTotal()` y `boolean guardar(Objeto o)`, que solo guarda si no se pasa de la capacidad.

Objeto	Gramos	Objeto	Gramos

- ☐ Al menos un objeto queda por fuera, y lo imprimen después: sigue existiendo.
- ☐ Explican al docente por qué es agregación y no herencia.



30
XP



Reto 5.2

Sella la arena

Con el ejemplo base 10, prueben tres trampas, una a la vez: heredar de `Reglas`, cambiar `VIDA_MAXIMA` desde `main` y sobrescribir `getNombre()` en una hija. Copien la parte clave de cada error y vuelvan a comentar la trampa.

- 1 _____
- 2 _____
- 3 _____

☐ Tres mensajes distintos del compilador.



Reto 5.3 Extra

Líder del escuadrón

Método `Personaje masFuerte()` en `Escuadron`: devuelve el miembro con más vida, o `null` si el escuadrón está vacío. En `main`, llámenlo primero con el escuadrón vacío sin que el programa explote, y luego con tres miembros.

- ☐ Revisan `null` antes de usar el resultado.
- ☐ Con tres miembros imprimen el nombre del líder.

Cazabugs

Main.java (tiene 2 bugs)

```
1 final class Reglas {
2     public static final int MAX_MIEMBROS = 3;
3 }
4
5 class Escuadron {
6     private ArrayList<Personaje> miembros;
7
8     public boolean agregar(Personaje p) {
9         if (miembros.size() > Reglas.MAX_MIEMBROS) {
10             return false;
11         }
12         miembros.add(p);
13         return true;
14     }
15 }
16
17 // en main
18 Escuadron e = new Escuadron();
19 System.out.println(e.agregar(new Personaje("A", 10)));
20 System.out.println(e.agregar(new Personaje("B", 10)));
21 System.out.println(e.agregar(new Personaje("C", 10)));
22 System.out.println(e.agregar(new Personaje("D", 10)));
```

Encuentra y corrige

Debería imprimir true, true, true y false: el cuarto miembro no cabe. Primero explota al ejecutar; cuando lo arreglan, deja entrar a los cuatro. Escriban línea y corrección. Cada bug vale **+10 XP**.

1 _____

2 _____

Checkpoint del Nivel 5

Marca con honestidad	Lo domino	Casi	Ayuda
Explico la diferencia entre “es un” y “tiene un” con un ejemplo propio.	☐	☐	☐
Dibujo agregación y composición con el rombo correcto.	☐	☐	☐
Sé qué bloquea <code>final</code> en una variable, un método y una clase.	☐	☐	☐
Uso constantes <code>static final</code> para las reglas de un programa.	☐	☐	☐

XP del nivel

Firma del docente: bonus sin Oráculo

Firma: explicación oral

Batalla contra el Dragón Null

herencia + polimorfismo + agregación + final

Misión

Un dragón de 330 puntos de vida bloquea la salida del gremio. Tu escuadrón tiene tres héroes: una maga, un arquero que ustedes van a forjar y un guerrero con armadura. Programarán el combate por fases. Pero hay trampa: con el orden en que el escuadrón entra a la batalla, pierden. Tendrán que pensar como estrategas y demostrarlo antes de ejecutar.

Fases de la batalla

Fase 1. Lee el mapa	20
Fase 2. Forja al Arquero	40
Fase 3. Invoca al dragón	30
Fase 4. La batalla	30
Fase 5. Estrategia demostrada	80
Total	200

Reglas del combate

- ✓ Cada ronda atacan primero todos los héroes vivos; sus daños se suman.
- ✓ Si el dragón sigue vivo, golpea con 35 al **primer héroe vivo** de la lista.
- ✓ Cada tercera ronda su golpe es doble: 70.
- ✓ Guerrero: su armadura resta 10 a cada golpe que recibe.

Reloj de la batalla

3' Fase 1	5' Fase 2	4' Fase 3	5' Fase 4	8' Fase 5
--------------	--------------	--------------	--------------	--------------

👤 **Todos a la vez.** En esta batalla el Piloto cambia en cada fase.

Escuadrón

280 HP

Dragón Null

330 HP



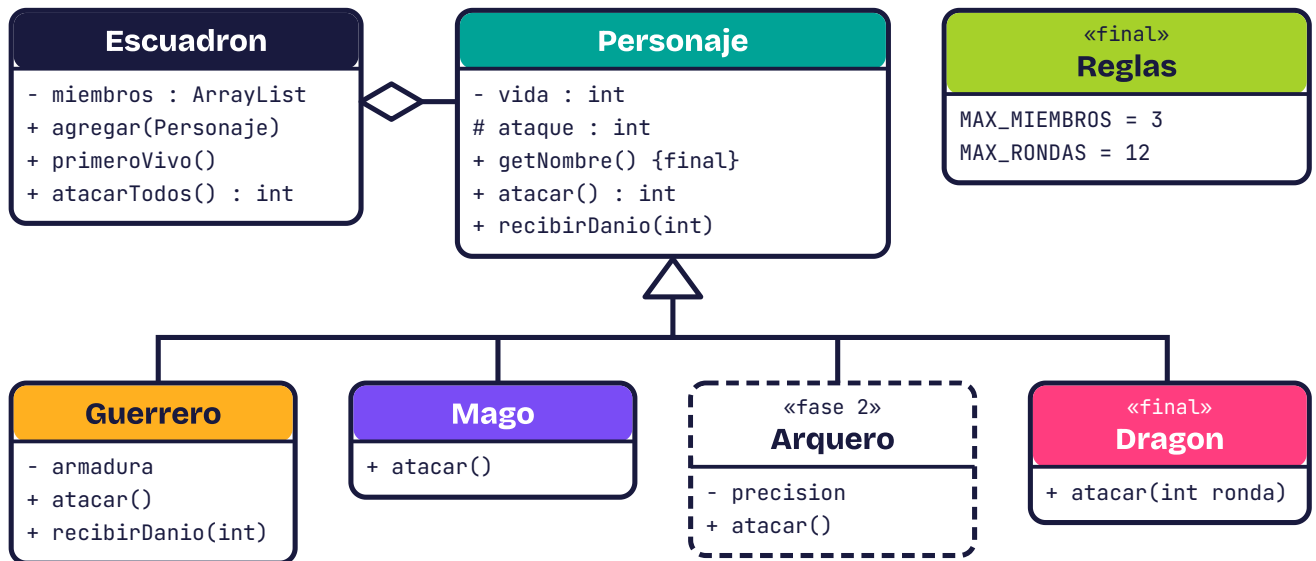
Antes de empezar: carguen la partida

Abran un `Main.java` nuevo y peguen completo el **punto de guardado** del Anexo C, que el docente comparte por el canal del curso. Trae `Reglas`, `Personaje`, `Guerrero`, `Mago` y `Escuadrón`: todo lo que construyeron en los niveles. Lo que escriban en las fases va debajo, en el mismo archivo.

Fase 1. Lee el mapa

+20 XP

Este es el diagrama de clases de la batalla. La caja punteada todavía no existe: la crean en la fase 2. Respondan mirando solo el diagrama.



Pregunta

Respuesta del equipo

¿Qué relación hay entre `Escuadrón` y `Personaje`, y qué símbolo la dibuja?

¿Qué dos métodos sobrescribe `Guerrero`?

¿Por qué conviene que `getNombre()` sea `final`?

Fase 2. Forja al Arquero

+40 XP

```

Main.java (debajo de todo)

1 class Arquero _____ Personaje {
2     private int precision;
3
4     public Arquero(String nombre, int vida,
5                     int ataque, int precision) {
6         _____(nombre, vida, ataque);
7         this.precision = precision;
8     }
9
10    _____
11    public int atacar() {
12        return _____;
13    }
14 }
  
```

Completen los cuatro espacios y copien la clase al final del archivo. Su atacar suma la precisión al ataque. Denle el nombre que quieran; en el solucionario se llama Aria.

Pregunta de los 40 XP: ¿por qué `Arquero` puede usar `getVida()` y `recibirDanio()` si no los escribió?

Fase 3. Invoca al dragón

+30 XP

```
Main.java (debajo del Arquero)

1  ----- class Dragon extends Personaje {
2      public Dragon() {
3          super("Dragón Null", 330, 35);
4      }
5
6      public int atacar(int ronda) {
7          if (ronda % 3 == 0) {
8              return ataque * 2;
9          }
10         return ataque;
11     }
12 }
```

Completen el espacio para que nadie pueda crear una subclase más débil del dragón. Luego respondan:

¿atacar(int ronda) sobrescribe o sobrecarga el atacar() de Personaje? _____

¿Qué responde el compilador si alguien escribe
class DragonBebe extends Dragon { }?

Fase 4. La batalla

+30 XP

Borren el comentario `// Fase 4` de `main` y escriban este código en su lugar. El Navegante verifica cada llave. Ejecuten con el orden original del escuadrón.

```
dentro de main

1  Escuadron escuadron = new Escuadron();
2  escuadron.agregar(new Mago("Lira", 70, 12));
3  escuadron.agregar(new Arquero("Aria", 90, 14, 6));
4  escuadron.agregar(new Guerrero("Brako", 120, 10, 10));
5
6  Dragon dragon = new Dragon();
7  int ronda = 0;
8
9  while (dragon.estaVivo() && escuadron.hayVivos()
10         && ronda < Reglas.MAX Rondas) {
11      ronda++;
12      int danio = escuadron.atacarTodos();
13      dragon.recibirDanio(danio);
14      System.out.println("Ronda " + ronda + ": daño " + danio
15                          + ". Dragón: " + dragon.getVida());
16
17      if (dragon.estaVivo()) {
18          Personaje objetivo = escuadron.primerVivo();
19          int golpe = dragon.atacar(ronda);
20          objetivo.recibirDanio(golpe);
21          System.out.println(" Golpe de " + golpe + " a "
22                              + objetivo.getNombre() + ". Vida: "
23                              + objetivo.getVida());
24      }
25  }
26
27  if (!dragon.estaVivo()) {
28      System.out.println("¡VICTORIA! en la ronda " + ronda);
29  } else if (!escuadron.hayVivos()) {
30      System.out.println("GAME OVER en la ronda " + ronda);
31  } else {
32      System.out.println("El dragón escapa al amanecer");
33  }
```

Qué combina este ciclo

-  **Agregación:** el escuadrón recibe héroes creados afuera.
-  **Polimorfismo:** `atacarTodos()` llama al `atacar()` de cada clase sin preguntar cuál es.
-  **Herencia y sobrescritura:** `objetivo.recibirDanio(golpe)` usa la armadura solo si el objetivo es un Guerrero.
-  **Final:** `Reglas.MAX_RONDAS` impide una batalla eterna.

Resultado con el orden original: _____
en la ronda _____

Cuando el dragón golpea a Brako con 35, ¿por qué su vida baja solo 25? ¿Qué método se ejecutó?

Fase 5. Estrategia demostrada

+80 XP

Cambien **solo el orden** de las tres líneas `escuadron.agregar(...)` para vencer al dragón. Los 80 XP se ganan únicamente si esta tabla está llena **antes** de ejecutar, el docente la firma y coincide con la consola. Si el dragón cae a mitad de una ronda, dejen vacías las columnas del golpe.

Nuestro orden: 1. _____ 2. _____ 3. _____

Ronda	Daño del escuadrón	Vida del dragón	Objetivo	Golpe	Vida Lira	Vida Aria	Vida Brako
inicio	—	330	—	—	70	90	120
1							
2							
3							
4							
5							
6							
7							
8							
9							

💡 Daño por ronda con los tres vivos: Lira 24, Aria 20, Brako 15. ¿Cuánto se pierde cuando cae cada uno?

💡 Las rondas 3, 6 y 9 son peligrosas: el golpe es 70. ¿Quién puede recibirlo sin caer?

💡 Pregunta clave: ¿a quién conviene proteger más tiempo, al que más daño hace o al que más resiste?

Firma del docente antes de ejecutar

Resultado en consola

XP de la fase



Modo difícil, bonus +30 XP (Extra)

Cambien el cuerpo de `atacar(int ronda)` del dragón por `return 25 + (int) (Math.random() * 21);`, que da un golpe entre 25 y 45. Ejecuten su estrategia cinco veces y anoten cuántas ganan: _____. ¿Por qué ahora la tabla ya no garantiza la victoria?

Después de la batalla

Discusión de 2 minutos, en voz alta, antes del cierre.

1. Si mañana el gremio agrega un `Sanador`, ¿qué partes del código de la batalla habría que cambiar? ¿Por qué tan pocas?
2. ¿Qué habría pasado si `vida` fuera `public` y alguien escribiera `dragon.vida = 0;`?
3. ¿Por qué el Dragón es `final` pero Personaje no puede serlo?

Créditos finales

10 min

3 min
Conteo de XP

4 min
Ticket de salida 3-2-1

3 min
Insignias y premiación

Ticket de salida 3-2-1

Individual, sin IA y sin mirar el cuadernillo de otro. Vale **+30 XP** para el equipo si todos lo entregan completo.

3	Tres conceptos de hoy que podría explicarle con un ejemplo propio a alguien que no vino.
2	Dos errores que ya no voy a cometer (por ejemplo, poner <code>void</code> en un constructor).
1	Una pregunta que todavía tengo sobre objetos, herencia o polimorfismo.

Insignias del taller

El docente marca la casilla de cada insignia ganada. Tómense una foto: es su pasaporte del gremio.



☐ **Forjador**

Completar los retos 1.1 a 1.3



☐ **Guardián de la bóveda**

Blindar la cuenta del gremio



☐ **Heredero**

Dibujar el árbol UML sin errores



☐ **Mil rostros**

Meter al Arquero en la arena sin tocar el ciclo



☐ **Sello final**

Provocar los tres errores de final



☐ **Cazador de Null**

Vencer al dragón con estrategia demostrada



☐ **Cazabugs**

Encontrar los 13 bugs del taller



☐ **Mente propia**

Terminar el taller sin usar el Oráculo



Premiación

El equipo con más XP recibe el título de Gran maestro POO y elige el tema del próximo taller. Hay reconocimiento aparte para la mejor explicación oral y para el equipo que más ayudó a otros.